



DSA Assignment - 2

Unit–2 Assignment: Linear Data Structures Toolkit (LDST)

Course Details

Name of the School: School of Engineering & Technology

Student Name: Tanuj

Section: A

Program (Class / Semester / Batch): B.Tech CSE (AI MI) / 2 /
2025-29

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Linear Data Structures

Faculty: Neetu Chauhan

Submission Date: 17/04/2026

1. Introduction

In this assignment, I implemented a Linear Data Structures Toolkit (LDST) in Python, including a Dynamic Array, Singly and Doubly Linked Lists, Stack, Queue, and a Balanced Parentheses Checker.

2. Dynamic Array: Complexity and Amortized Analysis

2.1 Idea of Dynamic Array

A dynamic array stores elements in a continuous block of memory and grows automatically when it becomes full.

When the array is full and we append a new element, it creates a new array of double capacity, copies all old elements, and then inserts the new one.

2.2 Time Complexity of Operations

- `append(x)`:
 - Normal case (no resize): $O(1)$ time, because we just place the element at the next free index.
 - Resize case: $O(n)$ time, because we copy n existing elements to a new array.
- `pop()`:
 - $O(1)$ time, because we only remove the last element and decrease the size.

2.3 Amortized Analysis of `append()`

In practice, most `append` operations are **constant** time, and only some are expensive when resizing happens.

When we double the capacity each time, the total work of copying across many appends spreads out evenly, so the average cost per `append` is $O(1)$ amortized.

In simple words: "Sometimes `append` is slow when `resize` happens, but because this does not happen very often, the average time per `append` stays constant."

3. Linked Lists: Complexity in Simple Words

3.1 Singly Linked List

A singly linked list stores elements in nodes, where each node points to the next node.

We implemented `insert at beginning`, `insert at end`, `delete by value`, and `traverse`. Complexities:

- `insert at beginning(x)`: $O(1)$, we just create a new node and adjust the head pointer.
- `insert at end(x)`: $O(n)$ if we do not maintain a tail pointer, because we traverse to the last node.
- `delete by value(x)`: $O(n)$, because we may need to scan the list to find x .
- `traverse()`: $O(n)$, we visit each node once to print all elements.

3.2 Doubly Linked List

A doubly linked list has nodes with both next and prev pointers, which makes forward and backward movement easier.

We implemented insert after node(target, x) and delete at position(pos).

Complexities:

- insert after node(target, x):
 - Finding target in worst case: $O(n)$, because we may scan the list.
 - Actual insertion after target: $O(1)$, we just update a few pointers.
- delete at position(pos):
 - Reaching the position: $O(n)$, because we traverse node by node.
 - Deleting that node: $O(1)$, updating prev and next links.

In simple words: Linked lists make insertion and deletion at known positions cheaper (no shifting like arrays), but searching is still linear.

4. Stack and Queue Using Singly Linked List

4.1 Stack ADT (LIFO)

The stack uses the head of the singly linked list for efficient operations.

Operations and complexities:

- push(x): $O(1)$, insert new node at head.
- pop(): $O(1)$, remove node from head (if not empty).
- peek(): $O(1)$, read the value at head without removing.

In simple words: Stack operations are constant time because we always work at one end (the top).

4.2 Queue ADT (FIFO)

The queue uses both head and tail pointers to get $O(1)$ operations.

Operations and complexities:

- enqueue(x): $O(1)$, insert new node at tail.
- dequeue(): $O(1)$, remove node from head.
- front(): $O(1)$, read the value at head.

In simple words: Queue operations are constant time because we add at the back (tail) and remove from the front (head) without any shifting.